

Scenario 1 — Search Functionality

Weak prompt : Test the search feature.

Output : Here's a structured QA test coverage for an e-commerce search feature, including functional + edge + usability scenarios.

Functional Test Cases

1. Valid Product Search (Positive)
 - Enter a valid product name (e.g., "iPhone")
 - Click search
 - Expected: Relevant product list is displayed
2. Partial Keyword Search
 - Enter partial text (e.g., "iph")
 - Expected: Matching products appear in results
3. Case Insensitivity
 - Search "IPHONE", "iphone", "iPhOnE"
 - Expected: Same consistent results returned
4. Different Product Types
 - Search across categories:
 - Electronics
 - Clothing
 - Accessories
 - Expected: Correct category-based results shown

Negative Test Cases

5. Empty Search
 - Click search without entering text
 - Expected: Message displayed:
 - "No results found"
6. Invalid Search Keyword
 - Enter random text like "@@##xyz"
 - Expected: No results found message
7. Special Characters Only
 - Input: "###\$\$%&"
 - Expected: No results displayed, proper validation message

Boundary Test Cases

8. Single Character Search
 - Input: "a"
 - Expected:
 - Typeahead suggestions appear
 - Suggestions are sorted alphabetically
9. Maximum Length Input
 - Enter very long string (1000+ chars)
 - Expected:
 - System should not crash
 - Either truncated input or safe handling

Typeahead / Autocomplete Validation

10. Typeahead Visibility
 - Enter "lap"
 - Expected: Suggestions dropdown appears
11. Alphabetical Order Validation

- Capture typeahead list
- Expected: Suggestions are sorted A → Z

12. Relevance Check

- Suggestions must be related to typed keyword only

UI / UX Validation

13. Search Button Functionality

- Verify clicking search icon triggers search

14. Enter Key Functionality

- Press Enter after typing
- Expected: Search executes

15. Result Highlighting

- Matching keywords should be highlighted in results (if supported)

Performance Test

16. Quick Response Validation

- Search execution should complete within acceptable SLA (e.g., <2s)

17. High Load Search

- Multiple rapid searches
- Expected: No lag or UI freeze

Security Test

18. Injection Attempt

- Input: `<script>alert(1)</script>`
- Expected: Input sanitized, no script execution

If you want, I can convert this into:

- Selenium Java POM automation framework
- TestNG test suite
- BDD (Cucumber) feature file
- Or add retry + reporting integration (Extent/Allure)

What's missing in the prompt :

1. The objective of the search is missing
2. The programming language and framework needed is missing
3. Product type
4. Positive, negative and boundary cases missing
5. No filter/sort behaviour
6. No empty/invalid input scenarios

Prompt using ICEPOT :

Context

Act as a QA Automation Engineer to write testscript for ecommerce website search functionality

Instructions

[MANDATORY] Generate Selenium Java Script with Selenium version 4.0 using page object model design pattern

[MANDATORY] Use TestNG annotations

[STRICT] Generate only the code no other text

[MANDATORY] Add Explicit waits

[CRITICAL] Do not add thread.sleep

[HIGHPRIORITY] Implement Retry mechanism

[HIGHPRIORITY] Implement Exception Handling

[MANDATORY] Add Assertions

[MANDATORY] Check for different type of products
[MANDATORY] "No results founds" should be displayed for empty search
[MANDATORY] Check if typahead is displayed based on the chars provided in search bar. Typahead should be in alphabetical order
[STRICT] Generate positive, negative and boundary cases
[LOWPRIORITY] Add comments to explain the code

Example:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
public class SalesforceLoginTest {
    WebDriver driver;
    WebDriverWait wait;
    @BeforeMethod
    public void setup() {
        driver = new EdgeDriver();
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }
    @Test
    public void verifySalesforceLogin() {
        driver.get("https://login.salesforce.com/");
        WebElement username = wait.until(
            ExpectedConditions.visibilityOfElementLocated(By.id("username"))
        );
        username.sendKeys("your_username");
        WebElement password = driver.findElement(By.id("password"));
        password.sendKeys("your_password");
        WebElement loginButton = driver.findElement(By.id("Login"));
        loginButton.click();
        wait.until(ExpectedConditions.or(
            ExpectedConditions.urlContains("lightning"),
            ExpectedConditions.urlContains("setup")
        ));
        Assert.assertTrue(
            driver.getCurrentUrl().contains("salesforce.com"),
            "Login failed or Salesforce home page not loaded"
        );
    }
    @AfterMethod
    public void tearDown() {
```

```
if (driver != null) {  
    driver.quit();  
}  
}  
}
```

Persona:

Act as a Automation Test Lead to Review the testscripts and give the Score

Output:

Code should be a downloadable text file

Tone:

Generate the output in a professional and concise manner, focusing on the technical aspects of the code.

Output of ICEPOT prompt :

```
import org.openqa.selenium.*;  
import org.openqa.selenium.chrome.ChromeDriver;  
import org.openqa.selenium.support.ui.ExpectedConditions;  
import org.openqa.selenium.support.ui.WebDriverWait;  
import org.testng.Assert;  
import org.testng.IRetryAnalyzer;  
import org.testng.ITestResult;  
import org.testng.annotations.*;
```

```
import java.time.Duration;  
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.List;
```

```
class RetryAnalyzer implements IRetryAnalyzer {
```

```
    private int currentRetry = 0;  
    private static final int MAX_RETRY_COUNT = 2;
```

```
    @Override
```

```
    public boolean retry(ITestResult result) {  
        if (currentRetry < MAX_RETRY_COUNT) {  
            currentRetry++;  
            return true;  
        }  
        return false;  
    }  
}
```

```
}
```

```
class BaseTest {
```

```
    protected WebDriver driver;  
    protected WebDriverWait wait;
```

```

@BeforeMethod
public void setup() {
    driver = new ChromeDriver();
    driver.manage().window().maximize();
    wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    driver.get("https://www.your-ecommerce-site.com");
}

@AfterMethod
public void tearDown() {
    if (driver != null) {
        driver.quit();
    }
}

}

class SearchPage {

    private WebDriver driver;
    private WebDriverWait wait;

    private final By searchBox = By.id("search");
    private final By searchButton = By.cssSelector("button[type='submit']");
    private final By productResults = By.cssSelector(".product-item");
    private final By noResultsMessage = By.xpath("//*[contains(text(),'No results found')]");
    private final By typeAheadSuggestions = By.cssSelector(".typeahead-suggestion");

    public SearchPage(WebDriver driver, WebDriverWait wait) {
        this.driver = driver;
        this.wait = wait;
    }

    public void enterSearchText(String searchText) {
        WebElement searchElement =
            wait.until(ExpectedConditions.visibilityOfElementLocated(searchBox));

        searchElement.clear();
        searchElement.sendKeys(searchText);
    }

    public void clickSearch() {
        wait.until(ExpectedConditions.elementToBeClickable(searchButton)).click();
    }

    public List<WebElement> getSearchResults() {
        return wait.until(ExpectedConditions.visibilityOfAllElementsLocatedBy(productResults));
    }
}

```

```

public boolean isNoResultsDisplayed() {
    try {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(noResultsMessage))
            .isDisplayed();
    } catch (TimeoutException e) {
        return false;
    }
}

public boolean isTypeAheadDisplayed() {
    try {
        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(typeAheadSuggestions))
            .isDisplayed();
    } catch (TimeoutException e) {
        return false;
    }
}

public List<String> getTypeAheadSuggestions() {

    List<WebElement> suggestions =
        wait.until(ExpectedConditions.visibilityOfAllElementsLocatedBy(typeAheadSuggestions));

    List<String> suggestionTexts = new ArrayList<>();

    for (WebElement suggestion : suggestions) {
        suggestionTexts.add(suggestion.getText().trim());
    }

    return suggestionTexts;
}

public class EcommerceSearchTest extends BaseTest {

    private SearchPage searchPage;

    @BeforeMethod
    public void initializePage() {
        searchPage = new SearchPage(driver, wait);
    }

    @Test(retryAnalyzer = RetryAnalyzer.class)
    public void verifySearchForElectronicsProduct() {

        try {

```

```

searchPage.enterSearchText("Laptop");
searchPage.clickSearch();

List<WebElement> results = searchPage.getSearchResults();

Assert.assertTrue(results.size() > 0,
    "No products returned for Laptop search");

} catch (Exception e) {
    Assert.fail("Exception occurred while searching electronics product: "
        + e.getMessage());
}
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifySearchForClothingProduct() {

    try {

        searchPage.enterSearchText("Shirt");
        searchPage.clickSearch();

        List<WebElement> results = searchPage.getSearchResults();

        Assert.assertTrue(results.size() > 0,
            "No products returned for Shirt search");

    } catch (Exception e) {
        Assert.fail("Exception occurred while searching clothing product: "
            + e.getMessage());
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifySearchForHomeApplianceProduct() {

    try {

        searchPage.enterSearchText("Refrigerator");
        searchPage.clickSearch();

        List<WebElement> results = searchPage.getSearchResults();

        Assert.assertTrue(results.size() > 0,
            "No products returned for Refrigerator search");

    } catch (Exception e) {

```

```

        Assert.fail("Exception occurred while searching appliance product: "
            + e.getMessage());
    }
}

```

```

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyInvalidProductSearch() {

```

```

    try {

```

```

        searchPage.enterSearchText("XYZINVALIDPRODUCT123");
        searchPage.clickSearch();

```

```

        Assert.assertTrue(
            searchPage.isNoResultsDisplayed(),
            "No Results message is not displayed for invalid product search");

```

```

    } catch (Exception e) {
        Assert.fail("Exception occurred during invalid product search: "
            + e.getMessage());
    }
}

```

```

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyEmptySearch() {

```

```

    try {

```

```

        searchPage.enterSearchText("");
        searchPage.clickSearch();

```

```

        Assert.assertTrue(
            searchPage.isNoResultsDisplayed(),
            "\"No results found\" message should be displayed for empty search");

```

```

    } catch (Exception e) {
        Assert.fail("Exception occurred during empty search validation: "
            + e.getMessage());
    }
}

```

```

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyBoundarySearchSingleCharacter() {

```

```

    try {

```

```

        searchPage.enterSearchText("A");

```

```

        Assert.assertTrue(
            searchPage.isTypeAheadDisplayed(),
            "Typeahead should be displayed for single character search");

    } catch (Exception e) {
        Assert.fail("Exception occurred for boundary test - single character: "
            + e.getMessage());
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyBoundarySearchMaximumCharacters() {

    try {

        String maxLengthSearch =
            "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZ";

        searchPage.enterSearchText(maxLengthSearch);
        searchPage.clickSearch();

        Assert.assertTrue(
            driver.getTitle() != null,
            "Application should handle max length search text");

    } catch (Exception e) {
        Assert.fail("Exception occurred during max length boundary test: "
            + e.getMessage());
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyTypeAheadSuggestionsDisplayed() {

    try {

        searchPage.enterSearchText("Lap");

        Assert.assertTrue(
            searchPage.isTypeAheadDisplayed(),
            "Typeahead suggestions are not displayed");

    } catch (Exception e) {
        Assert.fail("Exception occurred while validating typeahead display: "
            + e.getMessage());
    }
}

```

```

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyTypeAheadSuggestionsAlphabeticalOrder()
    try {

        searchPage.enterSearchText("Lap");

        List<String> actualSuggestions =
            searchPage.getTypeAheadSuggestions();

        List<String> expectedSortedSuggestions =
            new ArrayList<>(actualSuggestions);

        Collections.sort(expectedSortedSuggestions,
            String.CASE_INSENSITIVE_ORDER);

        Assert.assertEquals(
            actualSuggestions,
            expectedSortedSuggestions,
            "Typeahead suggestions are not displayed in alphabetical order");

    } catch (Exception e) {
        Assert.fail("Exception occurred while validating typeahead order: "
            + e.getMessage());
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifySpecialCharacterSearch() {

    try {

        searchPage.enterSearchText("@#$$%^&*");
        searchPage.clickSearch();

        Assert.assertTrue(
            searchPage.isNoResultsDisplayed(),
            "No Results message should be displayed for special character search");

    } catch (Exception e) {
        Assert.fail("Exception occurred during special character search: "
            + e.getMessage());
    }
}
}

```

Output 1: Manual test cases written in plain English. This is used for planning/testing design, not automation.

Output 2: Full Selenium TestNG automation framework using POM - executable code with waits, retries, assertions, and reusable structure.

Improvements :

- Converts test cases into runnable automation
- Adds framework structure (POM + BasePage)
- Includes retries, waits, and exception handling
- Adds real validations (products, typeahead, sorting)

Scenario 2 — File Upload

Weak prompt : Generate test cases for file upload.

Output : Here is a comprehensive set of test cases for a file upload feature, categorized by functionality, validation, security, performance, and usability.

Functional Test Cases

TC ID	Test Scenario	Expected Result
FU-001	Upload a valid file within size limit	File uploads successfully
FU-002	Upload multiple valid files (if supported)	All files upload successfully
FU-003	Upload file using drag-and-drop	File uploads successfully
FU-004	Upload file using file browser selection	File uploads successfully
FU-005	Cancel upload before completion	Upload stops and file is not saved
FU-006	Retry failed upload	Upload resumes/restarts successfully
FU-007	Remove uploaded file	File is deleted from the upload list
FU-008	Replace existing uploaded file	New file replaces old file
FU-009	Upload file after page refresh (if supported)	Upload persists or behaves as specified
FU-010	Upload same file multiple times	System handles duplicates according to requirements

File Type Validation

TC ID	Test Scenario	Expected Result
FU-011	Upload supported file type (.pdf)	Upload succeeds
FU-012	Upload supported image (.jpg, .png)	Upload succeeds
FU-013	Upload unsupported file type (.exe)	Error message displayed
FU-014	Upload file with no extension	Validation performed correctly
FU-015	Upload file with uppercase extension (.PDF)	Upload succeeds if allowed
FU-016	Upload file with double extension (file.pdf.exe)	Upload rejected
FU-017	Rename unsupported file to allowed extension	MIME validation prevents upload
FU-018	Upload corrupted file	Appropriate error displayed

TC ID	Test Scenario	Expected Result
FU-019	Upload zero-byte file	Upload rejected or handled per requirement

File Size Validation

TC ID	Test Scenario	Expected Result
FU-020	Upload file smaller than minimum size	Validation message shown if applicable
FU-021	Upload file exactly at maximum limit	Upload succeeds
FU-022	Upload file slightly below maximum limit	Upload succeeds
FU-023	Upload file slightly above maximum limit	Upload rejected
FU-024	Upload extremely large file	Upload rejected gracefully
FU-025	Upload multiple files exceeding total size limit	Validation message displayed

File Name Validation

TC ID	Test Scenario	Expected Result
FU-026	Upload file with normal name	Upload succeeds
FU-027	Upload file with spaces in name	Upload succeeds
FU-028	Upload file with special characters	Handled according to requirements
FU-029	Upload file with very long filename	Validation performed correctly
FU-030	Upload file with Unicode characters	Upload succeeds
FU-031	Upload file with leading/trailing spaces	Name handled properly
FU-032	Upload duplicate filename	System handles duplicates correctly

Security Test Cases

TC ID	Test Scenario	Expected Result
FU-033	Upload executable file (.exe)	Upload blocked
FU-034	Upload script file (.js, .sh, .bat)	Upload blocked
FU-035	Upload file containing malware	Upload blocked/quarantined
FU-036	Upload file with XSS payload in filename	Filename sanitized
FU-037	Upload file with SQL injection payload in filename	Input sanitized
FU-038	Attempt path traversal (../.././file.txt)	Upload rejected
FU-039	Modify upload request using proxy tools	Server-side validation prevents bypass
FU-040	Upload oversized file by tampering request	Server rejects request
FU-041	Upload malicious SVG with script content	Script execution prevented
FU-042	Unauthorized user uploads file	Access denied

Performance Test Cases

TC ID	Test Scenario	Expected Result
FU-043	Upload large file under normal network	Upload completes successfully
FU-044	Upload under slow network	Progress shown and upload completes
FU-045	Upload under intermittent network	Recovery/retry mechanism works

TC ID	Test Scenario	Expected Result
FU-046	Concurrent uploads by multiple users	System remains stable
FU-047	Upload maximum allowed number of files	Performance remains acceptable
FU-048	Stress test with high upload volume	No crashes or significant degradation

UI/UX Test Cases

TC ID	Test Scenario	Expected Result
FU-049	Verify upload progress indicator	Progress displayed correctly
FU-050	Verify success message after upload	Clear success notification shown
FU-051	Verify error message for invalid file	Meaningful error displayed
FU-052	Verify upload button state during upload	Appropriate enable/disable behavior
FU-053	Verify drag-and-drop area highlighting	Visual feedback displayed
FU-054	Verify uploaded file preview (if supported)	Preview displayed correctly
FU-055	Verify uploaded file metadata display	Correct filename, size, type shown

Compatibility Test Cases

TC ID	Test Scenario	Expected Result
FU-056	Upload using Chrome	Works correctly
FU-057	Upload using Firefox	Works correctly
FU-058	Upload using Safari	Works correctly
FU-059	Upload using Edge	Works correctly
FU-060	Upload from mobile devices	Works correctly
FU-061	Upload from different operating systems	Works correctly

API-Level Test Cases

TC ID	Test Scenario	Expected Result
FU-062	Verify upload API returns 200/201 on success	Correct response received
FU-063	Verify upload API returns validation error for invalid file	Appropriate error code returned
FU-064	Verify response contains uploaded file ID	File identifier returned
FU-065	Verify file metadata stored correctly	Metadata matches uploaded file
FU-066	Verify upload timeout handling	Proper timeout response
FU-067	Verify authentication token validation	Unauthorized requests rejected

These test cases cover most enterprise-grade file upload implementations and are suitable for web, mobile, and API testing.

What's missing in the prompt :

1. The programming language and framework needed is missing
2. Positive, negative and boundary cases missing
3. No file types
4. No size limits
5. No success/failure criteria

Prompt using ICEPOT :

Context

Act as a QA Automation Engineer to write testscript for ecommerce website file upload functionality

Instructions

[MANDATORY] Generate Selenium Java Script with Selenium version 4.0 using page object model design pattern

[MANDATORY] File type should be of either word or excel

[MANDATORY] File size should not exceed 2 MB

[MANDATORY] Empty file upload should not be accepted

[MANDATORY] Verify successful/unsuccessful file upload

[MANDATORY] Use TestNG annotations

[STRICT] Generate only the code no other text

[MANDATORY] Add Explicit waits

[CRITICAL] Do not add thread.sleep

[HIGHPRIORITY] Implement Retry mechanism

[HIGHPRIORITY] Implement Exception Handling

[MANDATORY] Add Assertions

[LOWPRIORITY] Add comments to explain the code

Example:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;

public class SalesforceLoginTest {
    WebDriver driver;
    WebDriverWait wait;

    @BeforeMethod
    public void setup() {
        driver = new EdgeDriver();
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }

    @Test
    public void verifySalesforceLogin() {
        driver.get("https://login.salesforce.com/");
        WebElement username = wait.until(
            ExpectedConditions.visibilityOfElementLocated(By.id("username"))
        );
        username.sendKeys("your_username");
        WebElement password = driver.findElement(By.id("password"));
        password.sendKeys("your_password");
        WebElement loginButton = driver.findElement(By.id("Login"));
```

```

loginButton.click();
wait.until(ExpectedConditions.or(
ExpectedConditions.urlContains("lightning"),
ExpectedConditions.urlContains("setup")
));
Assert.assertTrue(
driver.getCurrentUrl().contains("salesforce.com"),
"Login failed or Salesforce home page not loaded"
);
}
@AfterMethod
public void tearDown() {
if (driver != null) {
driver.quit();
}
}
}

```

Persona:

Act as a Automation Test Lead to Review the testscripts and give the Score

Output:

Code should be a downloadable text file

Tone:

Generate the output in a professional and concise manner, focusing on the technical aspects of the code.

Output of ICEPOT prompt :

```

package com.ecommerce.automation;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;
import org.testng.annotations.*;

import java.io.File;
import java.time.Duration;

/* Retry Mechanism */
class RetryAnalyzer implements IRetryAnalyzer {

    private int retry = 0;
    private static final int MAX_RETRY = 2;

```

```

@Override
public boolean retry(ITestResult result) {
    if (retry < MAX_RETRY) {
        retry++;
        return true;
    }
    return false;
}
}

/* Base Page */
class BasePage {

    protected WebDriver driver;
    protected WebDriverWait wait;

    public BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }

    protected WebElement waitVisible(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    protected WebElement waitClickable(By locator) {
        return wait.until(ExpectedConditions.elementToBeClickable(locator));
    }
}

/* File Upload Page */
class FileUploadPage extends BasePage {

    private final By fileInput = By.id("fileUploadInput");
    private final By uploadBtn = By.id("uploadBtn");
    private final By successMsg = By.id("uploadSuccess");
    private final By errorMsg = By.id("uploadError");

    public FileUploadPage(WebDriver driver) {
        super(driver);
    }

    /* Upload file */
    public void uploadFile(String path) {
        try {
            waitVisible(fileInput).sendKeys(path);
            waitClickable(uploadBtn).click();
        } catch (Exception e) {

```

```

        Assert.fail("File upload action failed: " + e.getMessage());
    }
}

public String getSuccessMessage() {
    return waitVisible(successMsg).getText();
}

public String getErrorMessage() {
    return waitVisible(errorMsg).getText();
}

/* Validate file type */
public boolean isValidFileType(File file) {
    String name = file.getName().toLowerCase();
    return name.endsWith(".doc") ||
           name.endsWith(".docx") ||
           name.endsWith(".xls") ||
           name.endsWith(".xlsx");
}

/* Validate file size <= 2MB */
public boolean isValidFileSize(File file) {
    long maxSize = 2 * 1024 * 1024;
    return file.length() <= maxSize;
}

/* Validate empty file */
public boolean isEmptyFile(File file) {
    return file.length() == 0;
}
}

/* Test Class */
public class EcommerceFileUploadTest {

    private WebDriver driver;
    private FileUploadPage uploadPage;

    @BeforeMethod
    public void setup() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://your-ecommerce-site.com/file-upload");

        uploadPage = new FileUploadPage(driver);
    }
}

```

```

/* POSITIVE CASE - VALID FILE UPLOAD */
@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyValidFileUpload() {

    try {
        File file = new File("C:\\TestData\\sample.docx");

        Assert.assertTrue(uploadPage.isValidFileType(file), "Invalid file type");
        Assert.assertTrue(uploadPage.isValidFileSize(file), "File exceeds 2MB");
        Assert.assertFalse(uploadPage.isEmptyFile(file), "File is empty");

        uploadPage.uploadFile(file.getAbsolutePath());

        Assert.assertTrue(
            uploadPage.getSuccessMessage().contains("successfully"),
            "Upload success message not displayed"
        );

    } catch (Exception e) {
        Assert.fail("Valid file upload test failed: " + e.getMessage());
    }
}

/* NEGATIVE CASE - INVALID FILE TYPE */
@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyInvalidFileType() {

    try {
        File file = new File("C:\\TestData\\image.png");

        Assert.assertFalse(uploadPage.isValidFileType(file), "Invalid file type accepted");

        uploadPage.uploadFile(file.getAbsolutePath());

        Assert.assertTrue(
            uploadPage.getErrorMessage().contains("Invalid file type"),
            "Error message not displayed for invalid file"
        );

    } catch (Exception e) {
        Assert.fail("Invalid file type test failed: " + e.getMessage());
    }
}

/* NEGATIVE CASE - FILE SIZE > 2MB */
@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyFileSizeExceeded() {

    try {
        File file = new File("C:\\TestData\\largeFile.xlsx");

```

```

        Assert.assertFalse(uploadPage.isValidFileSize(file), "Large file incorrectly accepted");
        uploadPage.uploadFile(file.getAbsolutePath());
        Assert.assertTrue(
            uploadPage.getErrorMessage().contains("File size exceeds"),
            "Size validation error not displayed"
        );

    } catch (Exception e) {
        Assert.fail("File size validation test failed: " + e.getMessage());
    }
}

/* NEGATIVE CASE - EMPTY FILE */
@Test(retryAnalyzer = RetryAnalyzer.class)
public void verifyEmptyFileUpload() {

    try {
        File file = new File("C:\\TestData\\empty.docx");

        Assert.assertTrue(uploadPage.isEmptyFile(file), "File is not empty");

        uploadPage.uploadFile(file.getAbsolutePath());

        Assert.assertTrue(
            uploadPage.getErrorMessage().contains("Empty file not allowed"),
            "Empty file upload was incorrectly accepted"
        );
    } catch (Exception e) {
        Assert.fail("Empty file upload test failed: " + e.getMessage());
    }
}

@AfterMethod
public void tearDown() {

    if (driver != null) {
        driver.quit();
    }
}
}

```

Output 1: Manual, enterprise-level *test case documentation* (what to test). Covers full scenarios: functional, security, performance, API, UX, compatibility.

Output 2: Selenium TestNG *automation framework* using POM (how to execute tests).

Improvements :

- Converts test cases into executable automation scripts
- Adds framework structure (POM + BasePage + Test Class)
- Implements: Retry mechanism, Explicit waits, Exception handling, Assertions
- Includes real validations: file type (Word/Excel only), file size $\leq$ 2MB, empty file rejection, success/failure verification

Scenario 3 — Shopping Cart

Weak prompt : Test the cart functionality.

Output : If you're referring to an e-commerce shopping cart, here is a concise test checklist for cart functionality:

Add to Cart

- Add a single item to the cart.
- Add multiple different items.
- Add multiple quantities of the same item.
- Verify the correct product, price, and quantity appear in the cart.
- Verify stock limits are enforced.

Update Cart

- Increase item quantity.
- Decrease item quantity.
- Set quantity to zero (if supported).
- Enter invalid quantities (negative numbers, text, very large values).
- Verify totals update correctly.

Remove Items

- Remove a single item.
- Remove all items.
- Verify cart totals and item counts update correctly.

Pricing & Calculations

- Verify subtotal calculation.
- Verify tax calculation.
- Verify shipping calculation.
- Verify discounts, coupons, and promotions.
- Verify rounding and currency formatting.

Persistence

- Refresh the page and confirm cart contents remain.
- Log out and log back in (if applicable).
- Verify cart behavior across multiple devices/browsers.

Navigation

- Continue shopping and return to the cart.
- Use browser back/forward buttons.
- Verify cart icon count updates correctly.

Checkout Integration

- Proceed to checkout with items in the cart.
- Verify cart contents match checkout contents.
- Verify out-of-stock items are handled correctly.

Edge Cases

- Add an out-of-stock product.
- Add discontinued or unavailable products.
- Test maximum quantity limits.
- Test concurrent updates from multiple tabs.

Performance & Security

- Verify cart loads quickly with many items.
- Verify unauthorized users cannot modify another user's cart.
- Verify pricing cannot be manipulated from the client side.

Expected Result

The cart should accurately maintain products, quantities, pricing, discounts, taxes, and shipping information while preserving data across supported user flows and preventing invalid operations. If this is for a specific website or application, I can create detailed test cases with steps, expected results, and priorities.

What's missing in the prompt :

1. The programming language and framework needed is missing
2. No pickup or delivery options mentioned
3. No add/remove/update actions
4. No price calculation validation
5. No coupon/discount scenarios
6. No out-of-stock handling.

Prompt using ICEPOT :

Context

Act as a QA Automation Engineer to write testscript for ecommerce website file upload functionality

Instructions

[MANDATORY] Generate Selenium Java Script with Selenium version 4.0 using page object model design pattern

[MANDATORY] Pickup or door delivery radio button should be selected

[MANDATORY] Discount should be calculated and the respective price should be displayed in the cart according to the quantity of products

[STRICT] Verify out of stock products are red in color

[MANDATORY] Use TestNG annotations

[STRICT] Generate only the code no other text

[MANDATORY] Add Explicit waits

[CRITICAL] Do not add thread.sleep

[HIGHPRIORITY] Implement Retry mechanism

[HIGHPRIORITY] Implement Exception Handling

[MANDATORY] Add Assertions

[LOWPRIORITY] Add comments to explain the code

Example:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;
import java.time.Duration;
public class SalesforceLoginTest {
    WebDriver driver;
    WebDriverWait wait;
    @BeforeMethod
    public void setup() {
        driver = new EdgeDriver();
        driver.manage().window().maximize();
```

```

wait = new WebDriverWait(driver, Duration.ofSeconds(15));
}
@Test
public void verifySalesforceLogin() {
driver.get("https://login.salesforce.com/");
WebElement username = wait.until(
ExpectedConditions.visibilityOfElementLocated(By.id("username")))
);
username.sendKeys("your_username");
WebElement password = driver.findElement(By.id("password"));
password.sendKeys("your_password");
WebElement loginButton = driver.findElement(By.id("Login"));
loginButton.click();
wait.until(ExpectedConditions.or(
ExpectedConditions.urlContains("lightning"),
ExpectedConditions.urlContains("setup")
));
Assert.assertTrue(
driver.getCurrentUrl().contains("salesforce.com"),
"Login failed or Salesforce home page not loaded"
);
}
@AfterMethod
public void tearDown() {
if (driver != null) {
driver.quit();
}
}
}

```

Persona:

Act as a Automation Test Lead to Review the testscripts and give the Score

Output:

Code should be a downloadable text file

Tone:

Generate the output in a professional and concise manner, focusing on the technical aspects of the code.

Output of ICEPOT prompt :

```
package com.ecommerce.automation;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.Color;
import org.openqa.selenium.support.ui.ExpectedConditions;

```

```

import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;
import org.testng.annotations.*;

import java.time.Duration;
import java.util.List;

/* Retry Mechanism */
class RetryAnalyzer implements IRetryAnalyzer {
    private int count = 0;
    private static final int MAX_RETRY_COUNT = 2;

    @Override
    public boolean retry(ITestResult result) {
        if (count < MAX_RETRY_COUNT) {
            count++;
            return true;
        }
        return false;
    }
}

/* Base Page */
class BasePage {
    protected WebDriver driver;
    protected WebDriverWait wait;

    public BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }

    protected WebElement waitForElement(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    protected void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
    }

    protected void type(By locator, String value) {
        WebElement element = waitForElement(locator);
        element.clear();
        element.sendKeys(value);
    }
}

```

```

/* Page Object */
class EcommerceUploadPage extends BasePage {

    private final By uploadInput = By.id("fileUpload");
    private final By uploadButton = By.id("uploadBtn");
    private final By uploadSuccessMessage = By.id("uploadSuccess");

    private final By quantityField = By.id("quantity");
    private final By discountedPrice = By.id("discountedPrice");
    private final By addToCartButton = By.id("addToCart");

    private final By pickupRadio = By.id("pickup");
    private final By deliveryRadio = By.id("doorDelivery");

    private final By outOfStockProducts = By.cssSelector(".out-of-stock");

    public EcommerceUploadPage(WebDriver driver) {
        super(driver);
    }

    public void uploadFile(String filePath) {
        try {
            waitForElement(uploadInput).sendKeys(filePath);
            click(uploadButton);

            Assert.assertTrue(
                waitForElement(uploadSuccessMessage).isDisplayed(),
                "File upload failed"
            );
        } catch (Exception e) {
            Assert.fail("Exception during file upload: " + e.getMessage());
        }
    }

    public void selectPickupOrDelivery() {
        try {
            WebElement pickup = waitForElement(pickupRadio);
            WebElement delivery = waitForElement(deliveryRadio);

            if (!pickup.isSelected() && !delivery.isSelected()) {
                pickup.click();
            }

            Assert.assertTrue(
                pickup.isSelected() || delivery.isSelected(),
                "Neither Pickup nor Door Delivery radio button is selected"
            );
        }
    }
}

```

```

    } catch (Exception e) {
        Assert.fail("Radio button selection failed: " + e.getMessage());
    }
}

public void validateDiscountCalculation(int quantity, double unitPrice, double discountPercentage)
{
    try {
        type(quantityField, String.valueOf(quantity));

        double expectedPrice =
            (unitPrice * quantity) -
            ((unitPrice * quantity) * (discountPercentage / 100));

        String displayedValue =
            waitForElement(discountedPrice).getText().replace("$", "").trim();

        double actualPrice = Double.parseDouble(displayedValue);

        Assert.assertEquals(
            actualPrice,
            expectedPrice,
            0.01,
            "Discounted price calculation mismatch"
        );

    } catch (Exception e) {
        Assert.fail("Discount validation failed: " + e.getMessage());
    }
}

public void addProductToCart() {
    try {
        click(addToCartButton);
    } catch (Exception e) {
        Assert.fail("Unable to add product to cart: " + e.getMessage());
    }
}

public void verifyOutOfStockProductsAreRed() {
    try {
        List<WebElement> products =
            wait.until(ExpectedConditions.visibilityOfAllElementsLocatedBy(outOfStockProducts));

        for (WebElement product : products) {

```

```

        String colorValue = product.getCssValue("color");
        String hexColor = Color.fromString(colorValue).asHex();

        Assert.assertTrue(
            hexColor.equalsIgnoreCase("#ff0000")
            || hexColor.equalsIgnoreCase("#dc3545")
            || hexColor.equalsIgnoreCase("#d32f2f"),
            "Out of stock product is not displayed in red color"
        );
    }

    } catch (TimeoutException e) {
        Assert.fail("No out-of-stock products found");
    } catch (Exception e) {
        Assert.fail("Color verification failed: " + e.getMessage());
    }
}

/* Test Class */
public class EcommerceFileUploadTest {

    private WebDriver driver;
    private EcommerceUploadPage ecommercePage;

    @BeforeMethod
    public void setup() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://your-ecommerce-application.com");

        ecommercePage = new EcommerceUploadPage(driver);
    }

    @Test(retryAnalyzer = RetryAnalyzer.class)
    public void verifyFileUploadAndCartFunctionality() {

        try {

            // Upload file
            ecommercePage.uploadFile("C:\\TestData\\product-image.png");

            // Verify Pickup or Door Delivery selection
            ecommercePage.selectPickupOrDelivery();

```

```

// Validate discount based on quantity
ecommercePage.validateDiscountCalculation(
    3,
    100.00,
    10.00
);

// Add item to cart
ecommercePage.addProductToCart();

// Verify out of stock products are red
ecommercePage.verifyOutOfStockProductsAreRed();

Assert.assertTrue(true, "Test completed successfully");

} catch (Exception e) {
    Assert.fail("Test execution failed: " + e.getMessage());
}
}

@AfterMethod
public void tearDown() {

    if (driver != null) {
        driver.quit();
    }
}
}

```

Output 1: Manual QA checklist for shopping cart functionality. It describes what to test (non-executable, no automation code).

Output 2: Selenium TestNG automation framework using Page Object Model. It implements how to test it in code.

Improvements :

- Converts checklist → executable automation script
- Adds framework structure (POM + BasePage + Test Class)
- Implements: Explicit waits, Retry mechanism, Exception handling, Assertions
- Adds real functional validations: Discount calculation logic, Cart operations, Out-of-stock product color verification, Radio button selection validation